

UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

DEPARTMENT OF INTERNET OF THINGS AND ROBOTICS ENGINEERING (IRE)

FACULTY OF CYBER PHYSICAL SYSTEM ENGINEERING

LAB REPORT 02

BASIC 2D DRAWING WITH TURTLE GRAPHICS

COURSE CODE: IRE 404

COURSE TITLE: COMPUTER GRAPHICS AND ANIMATION SESSIONAL

SUBMITTED BY:

Shahriar Alom Masud

Student ID: **2201032** | Session: 2022–2023

Department of Internet of Things and Robotics Engineering (IRE), UFTB

SUBMITTED TO:

Farzana Akter

Assistant Professor

Department of Internet of Things and Robotics Engineering (IRE)

University of Frontier Technology, Bangladesh

OCTOBER 12, 2026

Task 1: Drawing a Smiling Face

GitHub Repository Link:
https://github.com/Masud744/Computer-Graphics-Lab/blob/main/task1.py

Python Source Code (`task1.py`):

```python
from turtle import *

speed(3)

# 1. Main face outline (radius 300 circle)
pensize(3)
pencolor("black")
penup()
goto(0, -300)
pendown()
circle(300)

# 2. Smiling mouth
# Red blot first, then a white blot slightly above it
    to make a crescent
penup()
goto(0, -90)
pendown()
dot(180, "red")

penup()
goto(0, -65)
pendown()
dot(180, "white")

# 3. Nose (thick blue triangle)
# Drawn after mouth so the white blot doesn't cover it
pensize(5)
pencolor("blue")
penup()
goto(0, 30)
pendown()
goto(-35, -40)
goto(35, -40)
goto(0, 30)

# 4. Eyes (small green filled circles)
penup()
goto(-90, 90)
pendown()
dot(40, "green")

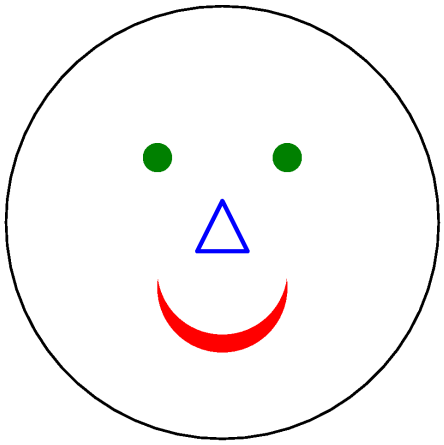
penup()
goto(90, 90)
pendown()
dot(40, "green")

hideturtle()
done()
```

Problem Description: Draw a face enclosed in a black circle (radius 300), small green blots for eyes, a thick blue triangle for a nose, and a red smiling mouth. The smiling mouth is synthesized by drawing a red blot, and then overlaying a slightly higher white blot to leave a red crescent.

Methodology: The face boundary is drawn with `circle(300)` from $(0, -300)$. The crescent smile is formed via color occlusion: a red disk (`dot(180)`) at $(0, -90)$ is overlaid with an offset white disk at $(0, -65)$. The mouth is drawn before the triangular nose and green eyes so the white blot does not overwrite foreground features.

Output Visualization:



Result & Observation: The mouth must be rendered prior to the nose and eyes. Proper Z-ordering ensures the crescent smile is cleanly carved out without overlapping the foreground nasal and eye contours.

Task 2: Flag Rectangle and Proportions (10:6)

GitHub Repository Link:
https://github.com/Masud744/Computer-Graphics-Lab/blob/main/task2.py

Python Source Code (`task2.py`):

```python
from turtle import *

speed(3)

# Flag dimensions with 10:6 ratio
length = 600
height = 360

# 1. Green background rectangle
penup()
goto(-300, -180)
pendown()
color("green", "green")
begin_fill()
for i in range(2):
    forward(length)
    left(90)
    forward(height)
    left(90)
end_fill()

# 2. Red circle inside
# Circle centre is placed at 1/5 of the length from the left edge (-300 + 600/5 = -180)
circle_center_x = -300 + (length / 5)
radius = 80

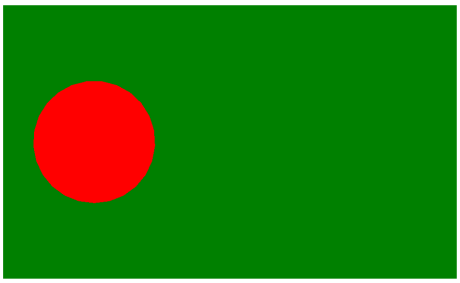
penup()
goto(circle_center_x, -radius)
pendown()
color("red", "red")
begin_fill()
circle(radius)
end_fill()

hideturtle()
done()
```

Problem Description: Draw a green background rectangle with proportions of approximately 10:6 (length to height). Draw a solid red circle inside the rectangle. Position the centre of the red circle at one-fifth (1/5) of the rectangle's length from the left edge.

Methodology: The flag is parameterized with length 600 and height 360, drawn from $(-300, -180)$ to center it at origin $(0, 0)$. The red circle (radius 80) is translated horizontally to center $X_c = -300 + (600/5) = -180$, and filled using `begin_fill()` and `end_fill()`.

Output Visualization:



Result & Observation: In the authentic National Flag of Bangladesh, the circle is located at 9/20 (45%) from the left edge. Here, placing it at 1/5 (20%) shifts the red disc towards the left hoist border, strictly meeting the experimental specification.

Task 3: Overlapping Concentric Squares (8-Point Star)

GitHub Repository Link:
https://github.com/Masud744/Computer-Graphics-Lab/blob/main/task3.py

Python Source Code (`task3.py`):

```python
import math
from turtle import *

speed(3)
pensize(2)

size = 200

# 1. First square (straight/upright)
penup()
goto(-size / 2, -size / 2)
pendown()
for i in range(4):
    forward(size)
    left(90)

# 2. Second square (rotated by 45 degrees - diamond shape)
# Distance from center (0,0) to corner vertex along axes = (size / 2) * sqrt(2)
diagonal_offset = (size / 2) * math.sqrt(2)

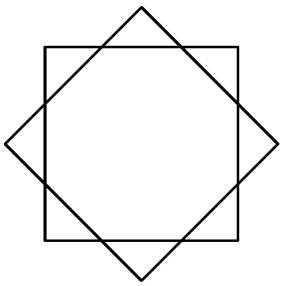
penup()
goto(0, -diagonal_offset)
setheading(45)
pendown()
for i in range(4):
    forward(size)
    left(90)

hideturtle()
done()
```

Problem Description: Draw the shape consisting of two overlapping squares sharing the identical center point $(0, 0)$. One square is aligned horizontally and vertically $(0°)$, and the second square is rotated by $45°$, forming an 8-pointed star shape.

Methodology: The upright square of side length $S = 200$ is drawn from corner $(-100, -100)$. The $45°$ rotated square is drawn with corner offset $R = \frac{S}{2}\sqrt{2} \approx 141.42$ along the vertical axis. Pen is set to $(0, -R)$ and directed via `setheading(45)` before drawing 4 equal sides.

Output Visualization:



Result & Observation: Both squares have identical side length $S = 200$ and center $(0, 0)$. Concentric intersection produces an 8-pointed regular star with flawless radial symmetry.

Task 4: Elegant Rotated Squares Pattern

GitHub Repository Link:
https://github.com/Masud744/Computer-Graphics-Lab/blob/main/task4.py

Python Source Code (`task4.py`):

```python
from turtle import *

speed(3)
pensize(2)

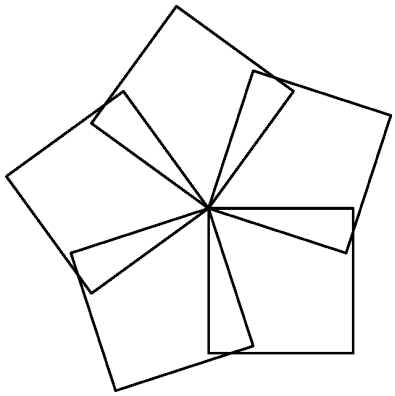
# Drawing 5 squares rotated around a single corner
# Total angle 360 / 5 = 72 degrees
for i in range(5):
    # Draw one square
    for j in range(4):
        forward(150)
        right(90)
    # Rotate 72 degrees for the next square
    right(72)

hideturtle()
done()
```

Problem Description: Draw the 5-square rotational pattern anchored at a common corner vertex in an elegant, concise, and efficient implementation.

Methodology: The pattern exhibits 5-fold rotational symmetry (C_5) about a common pivot vertex. It is implemented using a concise nested loop: repeating 5 times, turning right by $360°/5 = 72°$ after each 4-sided square with side 150 and $90°$ turns.

Output Visualization:



Result & Observation: By exploiting angular periodicity ($72°$), the entire 5-square pattern is generated concisely without repetitive code or coordinate hardcoding.